

Arquitetura de serviços em nuvem

ANDREWS EGAS

CAP 2

A disciplina

- Projetar, implementar e gerenciar infraestrutura de TI
- Integrar sistemas distribuídos e serviços em rede
- Garantir escalabilidade, segurança e qualidade dos sistemas

O 1º bimestre

- Introdução à computação em nuvem — conceitos e evolução
- On-Premise vs Cloud
- Modelos de serviço: IaaS, PaaS, SaaS, FaaS
- Arquitetura de integração e eventos
- Primeiros fluxos com n8n

Como vai funcionar

- Aulas expositivas com demonstração ao vivo
- Laboratórios práticos com Postman, n8n e Docker
- Trabalhos com foco em integração de serviços
- Projeto prático final — o PZaaS

Toda aula termina em algo funcionando.

Antes da nuvem

o reinado do On-Premise

O reinado do On-Premise

- Até o início dos anos 2000, tudo era On-Premise
- A empresa comprava servidor físico e instalava no próprio datacenter
- Ou numa sala com ar-condicionado improvisada
- A equipe de TI cuidava de tudo: hardware, SO, atualização, segurança, backup

Três problemas clássicos

- CAPEX alto — você comprava servidor para o pico (a Black Friday)
 - e ele ficava ocioso 90% do ano
- Escalabilidade lenta — precisou de mais? Espera semanas pelo hardware
- Complexidade e risco — servidor queimou ou link caiu, tudo parou

Esses três problemas são a razão de a nuvem existir.

O gatilho

o que empurrou o mercado

Explosão da internet

- Os anos 2000 trouxeram aplicações web massivas
- Google, Facebook e Amazon crescendo ao mesmo tempo
- A demanda passou a variar muito — às vezes milhões de usuários simultâneos

Startups e custos

- Empresas pequenas queriam competir, mas não tinham milhões para infraestrutura
- Precisavam de um jeito de:
 - pagar só pelo que usam
 - escalar rapidamente
 - não se preocupar com manutenção física

2006 — a Amazon

- Os times de desenvolvimento da Amazon perdiam tempo reconfigurando infraestrutura
- Criaram serviços internos padronizados:
 - servidores virtuais — hoje o EC2
 - armazenamento escalável — o S3
- Perceberam que aquilo podia ser vendido
- Em 2006 lançaram a AWS

A nuvem nasceu de um problema interno, não de um plano de mercado.

O modelo mental mudou

- Em vez de comprar servidor → você aluga quando precisa
- Em vez de pagar tudo antecipado → você paga pelo uso (OPEX)
- Em vez de cuidar de hardware → você foca no software e no negócio

O que aconteceu depois

- Escalabilidade imediata — o Instagram cresceu com 13 funcionários usando nuvem
- Inovação acelerada — ninguém mais esperava semanas por um servidor
- O mercado se reorganizou — hoje até banco e governo rodam em nuvem

Onde o mercado está hoje

- US\$ 143,4 bilhões movimentados só no 2º trimestre de 2026
- Crescimento de 43% em um ano — o maior em oito anos
 - AWS 28% · Azure 20% · Google Cloud 15%
- O que puxa esse crescimento hoje é carga de trabalho de IA

Fonte: Synergy Research, julho de 2026.

Uma frase para levar

- Arquitetura de serviços em nuvem: quando funciona, ninguém percebe
- Quando falha, está em todos os jornais e redes sociais
 - se elas estiverem funcionando

Arquitetura de serviços em nuvem

ANDREWS EGAS

CAP 3

O que vamos ver hoje

- Os três tipos de cloud: privada, pública e híbrida
- Os quatro modelos de serviço: IaaS, PaaS, SaaS, FaaS
- Quem gerencia o quê em cada modelo
- Técnicas para quando algum pedaço falhar

Tipos de Cloud

privada, pública e híbrida

Cloud Privada

- Infraestrutura dedicada a uma única organização
- No datacenter da empresa ou gerenciada por terceiros
 - Prós: mais controle e segurança
 - Contras: custo mais alto, manutenção própria
- Bancos — sistemas internos com dados críticos
- Hospitais — prontuário e exames, por segurança e LGPD

Cloud Pública

- Infraestrutura compartilhada entre várias empresas, isolada logicamente
- AWS, Azure, Google Cloud
 - Prós: custo menor, escalabilidade fácil
 - Contras: menos controle sobre o hardware
- Instagram e TikTok — nuvem pública (AWS e GCP)
- Fortnite — AWS para servidores de jogo no mundo todo
- Spotify — toda a infraestrutura no Google Cloud

Cloud Híbrida

- Combina pública e privada, integradas
- Dados sensíveis na privada, aplicação web na pública
 - Prós: flexibilidade e otimização de custo
 - Contras: integração mais complexa
- Netflix — roda o catálogo na AWS e entrega o vídeo pela Open Connect,
 - a CDN própria instalada dentro dos provedores de internet
- Governo Federal — sites em nuvem pública, dados sigilosos em privada

A Netflix é o melhor exemplo: infraestrutura própria posicionada na borda.

Modelos de Serviço

IaaS, PaaS, SaaS, FaaS

Os quatro modelos

- IaaS – Infrastructure as a Service
- PaaS – Platform as a Service
- SaaS – Software as a Service
- FaaS – Function as a Service (Serverless)
- A diferença entre eles é uma só: quanto o provedor faz por você

IaaS

Infrastructure as a Service

- Você aluga a infraestrutura. O resto é com você
- Servidores virtuais, rede e armazenamento sob demanda
- Você cuida do sistema operacional para cima
 - Prós: alta flexibilidade, controle e escalabilidade rápida
 - Contras: exige conhecimento técnico e mais responsabilidade
- AWS EC2 · Google Compute Engine · Azure VM

PaaS

Platform as a Service

- A plataforma está pronta. Você só sobe o código
- Ambiente de desenvolvimento e execução prontos
- Você cuida do código e dos dados; o provedor cuida do resto
 - Prós: menos configuração, escala automática, desenvolvimento rápido
 - Contras: menos controle, dependência do provedor
- Heroku · Google App Engine · Elastic Beanstalk · Vercel

SaaS

Software as a Service

- O software está pronto. É só usar
- Aplicações completas entregues pela internet
- O usuário só utiliza; o provedor gerencia tudo
 - Prós: nada para instalar, sempre atualizado, uso imediato
 - Contras: pouca personalização, dependência total do provedor
- Google Workspace · Slack · Trello · Canva · Dropbox

FaaS

Function as a Service · Serverless

- Você escreve funções que rodam sob demanda
- Executa código em resposta a eventos, sem gerenciar servidor
- Você cria só a lógica; o provedor cuida de toda a infraestrutura
 - Prós: paga só o tempo de execução, escala automática
 - Contras: latência inicial (cold start), não serve para processo longo
- AWS Lambda · Cloud Functions · Azure Functions

Quem gerencia o quê

- On-Premise você faz tudo
- IaaS provedor: hardware, rede, virtualização
- PaaS + sistema operacional, runtime, escalabilidade
- SaaS + a aplicação inteira
- FaaS você entrega só a função
- Quanto mais alto na escada, menos trabalho e menos controle

A pergunta que sobra: se der problema, de quem é a culpa?

Quando dá errado

porque vai dar

Desafios da computação em nuvem

- Segurança e privacidade dos dados
- Dependência de conectividade confiável
- Gerenciamento de custo e otimização de recursos
- Migração e integração de sistemas legados

O terceiro é hoje o assunto número um da profissão.

Técnicas essenciais

- Fallback — usar um recurso alternativo quando o principal falha
- Retry — repetir automaticamente a operação que falhou, após uma espera
- Circuit Breaker — parar de chamar um serviço que está falhando,
 - para não sobrecarregar o sistema inteiro

Retry sem cuidado piora a queda em vez de resolver.

Técnicas essenciais

parte 2

- Load Balancer — distribui requisições entre vários servidores
 - melhora desempenho e disponibilidade
- Cache — guarda dado temporariamente para não buscar de novo na fonte lenta

Chaos Monkey - Engenharia do Caos

- Quebrar de propósito, em produção, para descobrir o que não aguenta
- Nasceu na Netflix com o Chaos Monkey, que desligava servidores sozinho
- Hoje é prática de mercado: AWS Fault Injection, Gremlin, LitmusChaos
- O ritual chama Game Day — a equipe inteira reunida enquanto o sistema apanha

Vocês vão fazer isso no projeto final.

Para onde o mercado está indo

- IA generativa como carga de trabalho dominante — e escassez de GPU
- Serverless e computação na borda
- Nuvem soberana e residência de dados — no Brasil, puxada pela LGPD
- FinOps — a disciplina de controlar o custo da nuvem
- Internet das Coisas

Profissões em alta

- Arquiteto de Soluções Cloud
- Engenheiro DevOps · SRE · Platform Engineer
- Engenheiro de Segurança em Cloud
- Engenheiro de IA e Machine Learning
- FinOps Practitioner

Todas essas vagas pedem que você saiba ler um postmortem.

API e Webhook

na prática

Onde a gente chega hoje

- Pedir dados para qualquer sistema do mundo
- Receber aviso automático quando algo acontece lá
- Montar um fluxo que faz as duas coisas

API

Interface de Programação de Aplicações

- Application Programming Interface = contrato de comunicação
- Normalmente vem junto de uma Documentação
- Você pede (requisição) → ele processa (servidor) → traz a resposta
- REST troca dados em JSON (ou XML, mas quase sempre JSON)

Tipos de chamada

- GET → buscar dados
- POST → criar / enviar dados
- PUT / PATCH → atualizar
- DELETE → remover
- PUT troca o recurso inteiro. PATCH troca só um pedaço.

Ninguém decora API

Documentação

- Todo mundo lê documentação — inclusive quem usa a mesma API há anos
- O que procurar, nesta ordem:
 - Endpoint — a URL que você chama
 - Método — GET? POST? o que ela aceita
 - Parâmetros — o que dá para customizar
 - Resposta — quais campos voltam
 - Limites — quantas chamadas por minuto

Tipos de arquitetura de API

- REST → o meio-termo. APIs públicas, integração entre empresas
- gRPC → super performático (Google). Tráfego interno, alta escala
- GraphQL → mais controle sobre os dados, mais complexo. Front-end pesado
- SOAP → antigo, verboso, usa XML. Bancos e governo — e você vai encontrar

Não existe vencedor. Em entrevista a pergunta é: por que você escolheu esse?

Códigos de resposta

- 2xx → deu certo
- 4xx → a culpa é de quem chamou
- 5xx → a culpa é do servidor
 - 200 · 201 — OK · recurso criado
 - 400 — você mandou errado
 - 401 · 403 — sem credencial ou sem permissão
 - 404 — não existe
 - 429 — rate limit, você chamou vezes demais
 - 500 · 503 — erro interno · serviço indisponível

Postman

- Plataforma para desenvolver e testar APIs
- Interface gráfica: criar, testar, documentar e compartilhar
 - Params → os parâmetros da URL
 - Headers → informações extras da requisição
 - Body → o que você envia no POST
 - Response → o que voltou, e o código de status
- postman.com

APIs que vamos usar

- PokeAPI – `pokeapi.co/docs/v2`
- SpaceX – `docs.spacexdata.com`
- CoinDesk – `developers.coindesk.com`
- GitHub – `docs.github.com/rest`
- Todas gratuitas, sem cadastro e sem chave de acesso

Escolha UMA para o trabalho da semana.

Parâmetros comuns

- `?` inicia o envio de parâmetros (QueryParam)
 - `&` separa um parâmetro do outro
 - `limit` → quantos itens trazer (10, 50, 100)
 - `offset` → a partir de onde começar a buscar
-
- `https://pokeapi.co/api/v2/pokemon?limit=10&offset=21`
 - traga 10 registros a partir do 21º

Prática 1

resolva sozinho, com a documentação aberta

Quatro desafios

As requisições já estão na coleção — só faltam os valores

- Traga os Pokémon da posição 51 até a 60
- Descubra o peso e os tipos do Charizard
- Traga nome, data e resultado do último lançamento da SpaceX
- Chame o GitHub 5 vezes e olhe o header x-ratelimit-remaining

Webhook

- API → você vai até o servidor e pede a informação
- Webhook → o servidor te avisa sozinho quando algo acontece
- É uma requisição HTTP que o sistema de origem envia
- Para um endpoint (URL) que você configurou
- Ex.: o pedido do iFood sai da cozinha → o app avisa você automaticamente

Quem manda webhook para você

- GitHub → quando alguém faz push, abre PR ou issue
- Stripe / Mercado Pago → quando o pagamento é aprovado ou falha
- iFood → quando o pedido sai para entrega
- Slack / Discord → recebem mensagens via webhook (integrações)
- WhatsApp Business API → quando alguém te manda mensagem

Ver um webhook chegando

Demonstração ao vivo — 5 minutos

- Abra webhook.site — uma URL única aparece, sem cadastro
- Copie a URL e cole na variável da coleção
- Dispare o POST pelo Postman
- A requisição aparece na tela em tempo real, com headers e corpo

Guardem essa ideia: daqui a pouco o n8n cria uma URL dessas para vocês.

n8n

automação e integração de sistemas

n8n

- Ferramenta de automação e integração de sistemas, open source
- Um orquestrador visual: você conecta blocos chamados nodes
- Cada node é um serviço, uma API ou uma lógica
- Você liga um no outro e o dado flui — sem escrever código
- n8n.io · github.com/n8n-io/n8n · docs.n8n.io

Instalando

- `npm install n8n -g`
- `n8n start`
- Ou, sem instalar nada:
- `npx n8n`
- Depois abra: `http://localhost:5678`

Precisa do Node.js instalado. A primeira vez demora.

Os quatro nodes do nosso fluxo

- Webhook → o gatilho. Cria a URL que dispara o fluxo
- HTTP Request → chama a API pública
- Edit Fields (Set) → escolhe quais informações manter
- Respond to Webhook → devolve o JSON organizado

Esses quatro são exatamente o requisito mínimo do trabalho.

TP — API e Webhook

Requisito mínimo obrigatório

- Criar um gatilho inicial no n8n (não precisa expor a URL pública)
- Fazer uma requisição a uma API pública: PokeAPI, SpaceX, CoinDesk ou GitHub
- Processar a resposta retornando pelo menos 3 informações relevantes
- Retornar a saída em JSON organizado, usando um nó Respond to Webhook

EXTRA: utilizar um `queryParams` na chamada do webhook.

Entrega

- ENVIAR PARA: andrewsegas@unisanta.br
- ASSUNTO: TP-DATA-MATERIA-TURMA
- 1 — Arquivo JSON contendo o fluxo do n8n
- 2 — PDF com print do resultado, contendo NOME e RA

Na próxima

- ngrok — expor a sua URL para o mundo
- Redis — banco de dados em memória
- PZaaS — a especificação do nosso projeto

Tragam o n8n rodando.

Arquitetura de serviços em nuvem

ANDREWS EGAS

CAP 4

PZaaS

Pizza as a Service

O projeto

- Uma pizzeria inteira, construída em serviços separados
- Cada dupla é dona de UM serviço
- Nenhum serviço funciona sozinho. todos dependem dos outros
- No fim, a turma inteira é um sistema distribuído

Se um serviço cair, o pedido de alguém não sai. É assim lá fora também.

Como funciona

- Cada dupla publica a API do seu serviço
- Cada dupla consome a API de outros serviços
- Todos seguem o mesmo padrão de contrato
- Na última semana, tudo é ligado ao mesmo tempo

A sua nota depende de outra dupla conseguir usar o seu serviço.

Os serviços

Parte 1

- 01 API Gateway – ponto único de entrada, orquestra o checkout
- 02 Catálogo / Cardápio – lista pizzas, ingredientes, promoções
- 03 Disponibilidade / Estoque – confirma e atualiza insumos
- 04 Pagamento – aprova ou recusa, controla o caixa
- 05 Orquestração de Pedido – cria e evolui o status do pedido

Os serviços

Parte 2

- 06 Forno – simula o tempo de preparo e avança o status
- 07 Fila de Produção – enfileira e libera os pedidos prontos
- 08 Cadastro / Identidade – valida token e devolve o perfil
- 09 Logger – recebe logs estruturados e apresenta
- 10 Chaos Monkey – introduz falhas controladas

Serviços 02 a 05 existem em versão A e B. É o par para o fallback.

O serviço CLIENTE

- GET /cardapio → recebe a lista de pizzas
- POST /pedido → envia a escolha e o pagamento
- GET /status → acompanha o andamento
- POST /entrega → recebe o aviso de que saiu para entrega
- Os três primeiros ele CHAMA. O último ele RECEBE

O /entrega é um webhook. É a pizzeria ligando de volta para o cliente.

Padrões globais

O contrato que a turma inteira assina

- Versão de API: v1 nos endpoints – /v1/menu
- Protocolo: HTTP / JSON
- Healthcheck obrigatório: GET /health
- Valem para TODOS os serviços, sem exceção

Headers obrigatórios

- `Content-Type: application/json`
- `x-api-key: turma2026`
- `x-pedido-id: <id>` - Apenas pelos serviços que o pedido já foi gerado
- O `x-pedido-id` é gerado pelo Orquestrador ou Gateway e propagado por todos os serviços que já possuem um pedido gerado.
- É ele que permite rastrear um pedido atravessando todos os serviços

Resiliência

- `Retry` – até 2 tentativas com `backoff`, em `5xx` ou `timeout`
- `Fallback` – se A falhar, tentar B
- Por isso temos redundância dos serviços.
- Se o Cardápio A cair, o Gateway vai no Cardápio B

É a diferença entre o sistema cair e o cliente nem perceber.

Erros padronizados

- 200 OK
- 400 Bad Request – payload inválido
- 401 / 403 sem x-api-key ou inválida
- 404 recurso não encontrado
- 429 rate limit (Gateway)
- 500 erro interno
- 503 serviço indisponível, após retries e fallback

Observabilidade

- Todos enviam log para o Logger – POST /v1/logs
- De forma assíncrona, sempre que possível
- Sem isso, quando algo quebrar ninguém descobre onde

O entregável

em dupla · três partes

Três entregas

- Documentação da API
- Apresentação em produção
- Apresentação oral
- Em dupla. Os dois fazem, os dois falam, os dois respondem

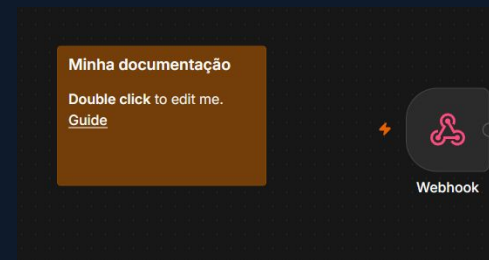
As três valem. Não adianta ir bem em duas.

1 · Documentação da API

- O contrato do seu serviço, escrito para outra pessoa usar
 - Endpoints, método e versão
 - Headers obrigatórios
 - Corpo da requisição e da resposta
 - Todos os códigos de erro que o serviço devolve
 - Pelo menos um exemplo de chamada REAL, com resposta real

O critério da documentação

- Outra dupla precisa conseguir integrar com o seu serviço
- Lendo só a documentação
- Sem te perguntar nada
- Sem olhar o seu fluxo no n8n
- Deixe o Link da documentação no seu fluxo do N8N em um stick note:
- Deixe o Link da documentação em um comentário na [planilha](#)



É esse o teste. Se precisou perguntar, a documentação não está pronta.

Onde documentar

- Postman — Collection → Publish Docs, gera uma página pública
- Ou README no GitHub, se preferir
- Vale qualquer formato, desde que tenha link e esteja no ar
- O que não vale: print, arquivo solto, ou "está no meu computador"

Postman gerando documentação é o caminho mais rápido — vocês já sabem usar.

2 · Apresentação em produção

- O seu serviço no ar, durante a apresentação
 - URL acessível pelos outros grupos
 - GET /health respondendo
 - Recebendo chamada ao vivo de outra dupla
 - Enviando log para o Logger

"Funciona na minha máquina" não conta como produção.

3 · Apresentação oral

- OITO minutos por dupla
- Os dois falam. Não vale um apresentar e o outro assistir
- Eu pergunto para quem eu escolher
- Não saber explicar uma decisão do próprio serviço zera o item

Por onde começar

- Escolham o serviço da dupla hoje
- Leiam a documentação do projeto inteiro, não só a do seu serviço
- Descubram quem chama vocês e quem vocês chamam
- Combinem o contrato com essas duplas ANTES de construir
- Subam o /health primeiro - é o menor pedaço que já prova que está no ar

Quem combina o contrato na primeira semana termina o projeto sem susto.

Banco de dados - Redis (cloud) - opcional

Redis é um banco de dados em memória, rápido, usado para guardar informações que precisam ser acessadas com baixa latência (milissegundos).

- Criar uma conta free : <https://cloud.redis.io/>
- Dentro do N8N ao adicionar o nó Redis insira as credenciais:
- Utilizar a URL informada no site do Redis.
 - Utilize a porta informada após a URL.
 - Utilizar a senha informada no site do Redis.
 - Detalhes visuais na próxima página

Banco de dados - Redis (cloud) - opcional

N8N

The screenshot shows the 'Redis account' configuration page. The 'Connection' tab is active, displaying a green message: 'Connection tested successfully'. Below this, there's a section for 'Need help filling out these fields? Open docs'. The 'Password' field is highlighted with a blue box. The 'Host' field is highlighted with a red box and contains the value 'redis-10757.crce181.sa-east-1-2.ec2.redns.redis-cloud.com'. The 'Port' field is highlighted with a green box and contains the value '10757'. The 'Database Number' is set to '0'.

The screenshot shows the 'General' configuration page for a Redis instance. The 'Database name' is 'database-andrews'. The 'Public endpoint' is 'redis-10757.crce181.sa-east-1-2.ec2.redns.redis-cloud.com' with the port '10757' highlighted in a green box. A 'Connect' button is visible at the bottom.

Site do Redis

The screenshot shows the 'Security' configuration page for a Redis instance. The 'Default user' is set to 'On (Username: "default")'. The 'Default user password' field is highlighted with a blue box and contains a masked password. A 'Copy' button is visible next to the password field.

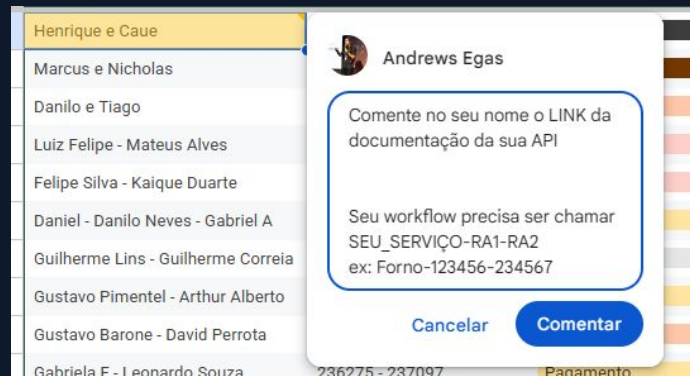
Link de acesso aos serviços - N8N Publico - Obrigatório

- Planilha: [link aqui](#)
- Crie sua **SENHA** com o link que está ao lado do SEU grupo/nomes na planilha
- Deixe o **comentário** na planilha com o link do da sua documentação. ex:
- Seu workflow precisa chamar

Seu_Serviço-RA1-RA2

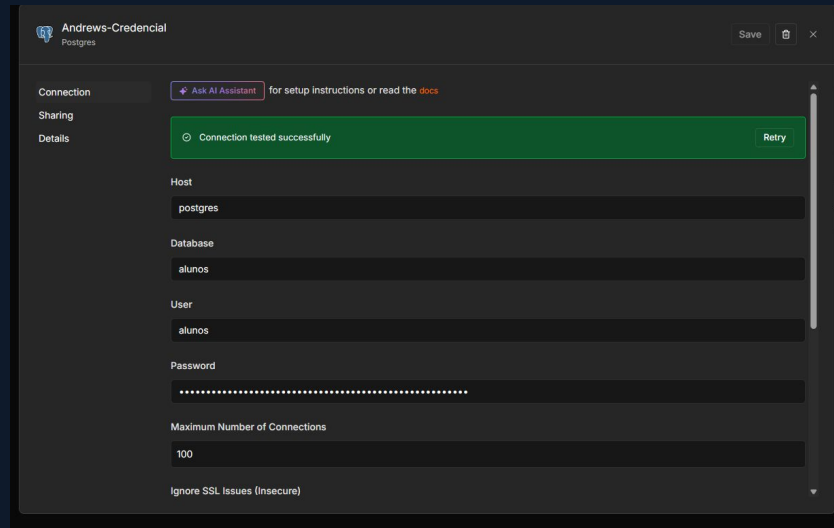
ex: *Forno-12345-23456*

- **ALTERE APENAS O SEU WORKFLOW**



Se precisar utilizar Postgres - opcional

- Host :postgres
- Database :alunos
- User :alunos
- Password :19b85000344560d6d90aa84424bf82933cb085850f2f3ab9
- Port :5432



Se precisar utilizar Postgres - opcional

- **SEMPRE** que criarem uma tabela
- É **obrigatório** utilizar o prefixo da dupla nas tabelas
- ex para uma tabela de pedidos:
 - turmab01_pedidos

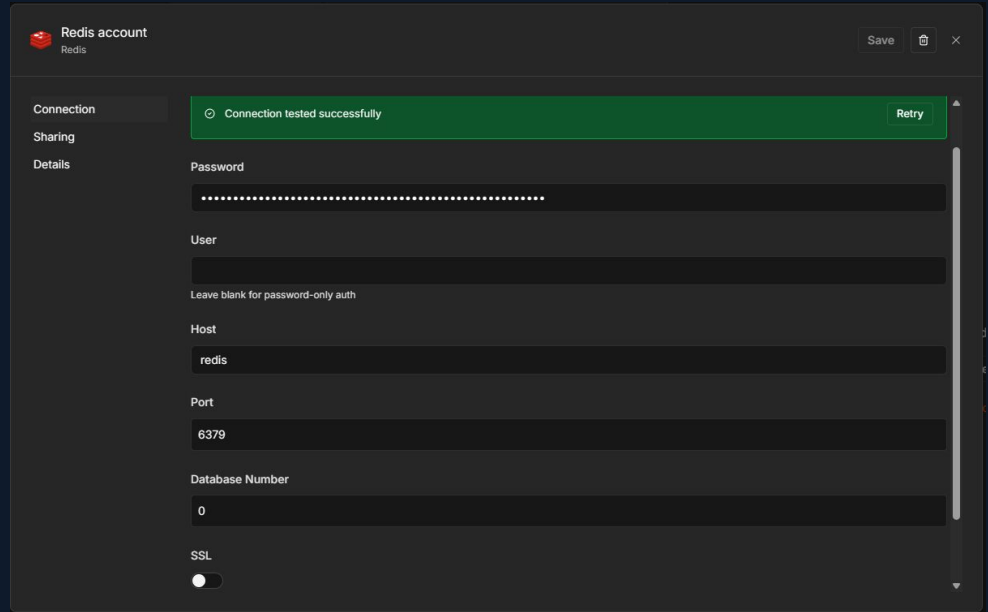
The screenshot shows a database management tool interface with two tabs: "Parameters" and "Settings". The "Parameters" tab is active. It contains a section for "Credential" with a dropdown menu showing "Andrews-Credencial" and an edit icon. Below this is the "Operation" section with a dropdown menu showing "Execute Query". The "Query" section contains a text area with the following SQL code:

```
1 CREATE TABLE IF NOT EXISTS turmab01_pedidos (  
2     id          BIGSERIAL PRIMARY KEY,  
3     cliente     TEXT NOT NULL,  
4     sabor       TEXT NOT NULL,  
5     status      TEXT NOT NULL DEFAULT 'recebido',  
6     valor       NUMERIC(10,2),  
7     criado_em   TIMESTAMPTZ NOT NULL DEFAULT now()  
8 );  
9  
10 CREATE INDEX IF NOT EXISTS idx_turmab01_pedidos_status  
11     ON turmab01_pedidos (status);
```

Below the query text area, there is a warning: "Consider using query parameters to prevent SQL injection attacks. Add them in the options below". Under the "Options" section, it says "No properties" and there is an "Add option" button with a dropdown arrow.

Para utilizar o Redis (do nosso n8n)

- Password :ad0e8da2695c0142de4f780b1cd57d278908df37415c14f2
- User :
- Host :redis
- Port :6379
- Database :0
-



The screenshot shows a 'Redis account' configuration window. On the left, there are tabs for 'Connection', 'Sharing', and 'Details'. The 'Connection' tab is active, displaying a green success message: 'Connection tested successfully' with a 'Retry' button. Below this, the 'Details' section contains several input fields: 'Password' (masked with dots), 'User' (empty), a note 'Leave blank for password-only auth', 'Host' (filled with 'redis'), 'Port' (filled with '6379'), 'Database Number' (filled with '0'), and an 'SSL' toggle switch which is currently turned off.